

Project: QUIC Traffic Identification, Classification & Policing

WEEK-1 TASK:

1. Deep dive into QUIC protocol (RFC 9000, 9001): handshake, connection IDs, version negotiation
2. Study differences between QUIC and generic UDP at the wire level
3. Set up traffic generation environment (quiche, ngtcp2, Chrome/Firefox QUIC)
4. Capture QUIC and non-QUIC UDP traffic samples
5. Study existing QoS and policing mechanisms

NAME: KAVYA JAIN

REG NO: 23BIT0006

QUIC

QUIC is a UDP-based transport protocol that combines security, reliability, and multiplexing to provide low-latency, encrypted communication for modern Internet applications such as HTTP/3.

Why QUIC is better :

- **Faster connection setup** – 1-RTT handshake, supports **0-RTT**
- **No head-of-line blocking** – packet loss in one stream doesn't block others
- **Always encrypted** – built-in TLS 1.3
- **Runs over UDP** – easier updates, faster protocol evolution
- **Connection migration** – survives IP/network changes (Wi-Fi ↔ mobile)
- **Better for video & web** – lower latency, smoother performance
- **Modern web standard** – used by HTTP/3 (Chrome, YouTube, Google)

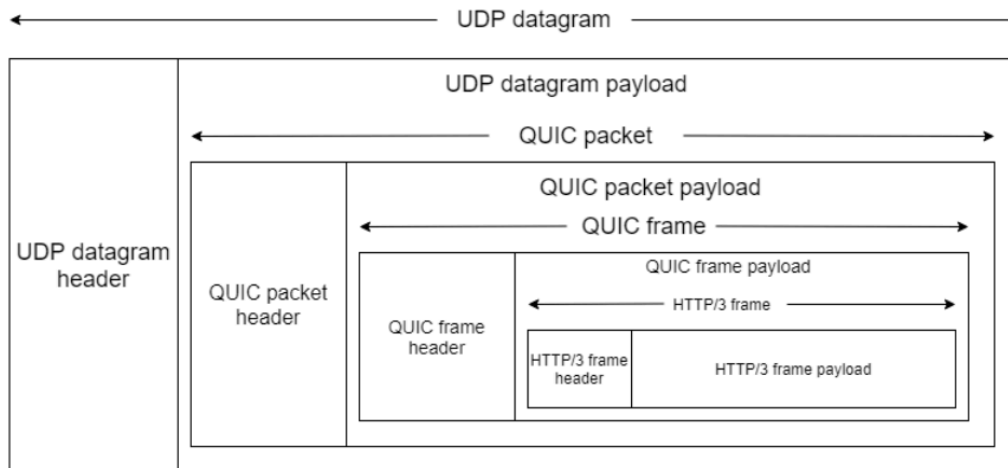
Scenario: A client wants to access a resource hosted on a server using its **domain name** (e.g., www.youtube.com) over QUIC.

The process consists of:

1. DNS resolution (domain name → IP address)
2. QUIC connection establishment
3. TLS 1.3 handshake over QUIC
4. Secure application data transfer

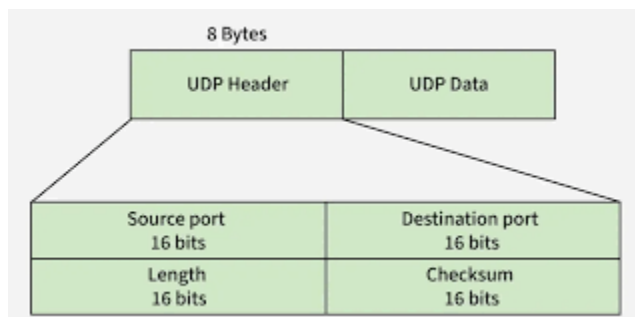
Step 1: DNS Resolution (Domain Name → IP Address)

Step 2: Client Initiates QUIC Connection (Initial Packet)



UDP Packet (Transport Layer):

- UDP Header (8 bytes total)



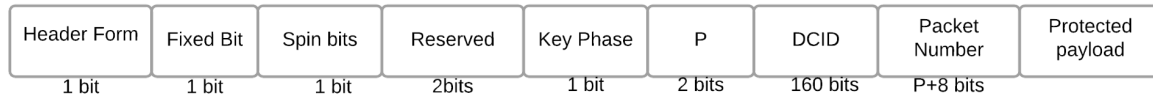
- UDP Payload = QUIC Initial Packet (QUIC header + payload)

1) QUIC Header :

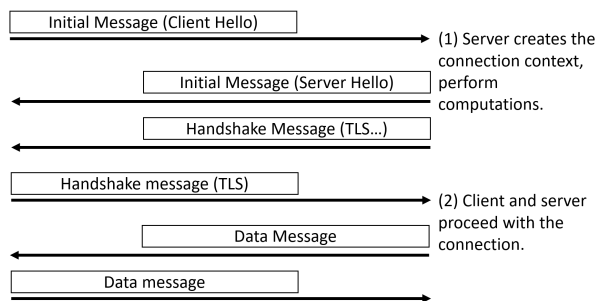
Long Header



Short Header



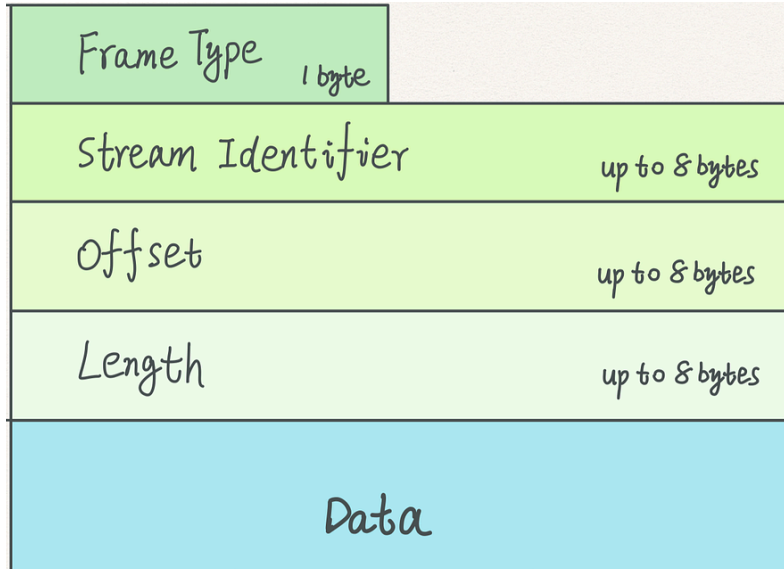
DCID (Destination Connection ID) and SCID (Source Connection ID), which together enable connection identification independent of IP addresses, allowing connection migration.



2) QUIC Initial Payload (Encrypted) :

- Payload is encrypted using **Initial keys**
- Initial keys derived from:
 - Destination Connection ID (DCID)
 - Fixed Initial Salt (RFC 9001)
- Anyone can derive these keys, but encryption prevents ossification
- Structure: Crypto frames + padding frames

QUIC frame:



- Crypto Frame contains **TLS handshake bytes** as data
- Initial QUIC packet sends ClientHello in crypto frame

ClientHello:

- Supported TLS versions (TLS 1.3)
- Supported cipher suites
- Supported signature algorithms
- Supported key exchange groups
- **KeyShare** (client's ephemeral Diffie–Hellman public key)
- Server Name Indication (SNI)
- Application-Layer Protocol Negotiation (ALPN, e.g., h3)
- QUIC transport parameters
- Legacy session ID (for compatibility)
- Random value (client random)
- Compression methods (null)

Step 3: Server Processes Client Initial Packet

Server Actions

1. Receives UDP packet

2. Parses QUIC Initial header
3. Extracts DCID
4. Derives Initial keys
5. Decrypts QUIC payload
6. Extracts CRYPTO frame
7. Reassembles TLS ClientHello bytes
8. Passes ClientHello to TLS stack

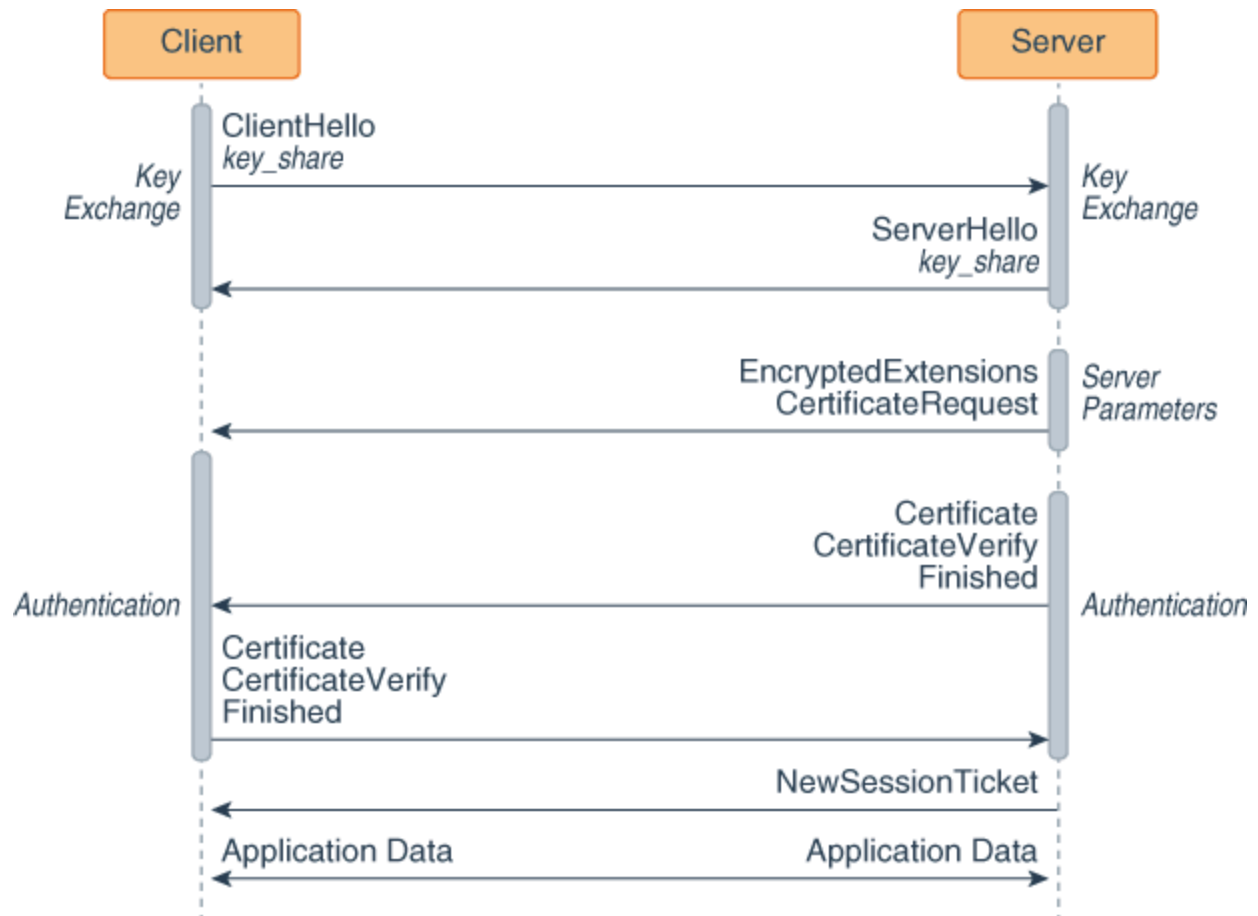
Step 4: Server Responds with ServerHello (similar packet is sent as sent in step two)

TLS ServerHello Contents

- Selected TLS version
- Selected cipher suite
- Server's ephemeral Diffie–Hellman public key

Step 5: Server Sends Handshake Messages (Authentication Phase)

The server now sends **QUIC Handshake packets**, encrypted using **Handshake keys**.



Step 5.1: EncryptedExtensions

Contains:

- Selected ALPN (e.g., h3)
- QUIC transport parameters

Step 5.2: Certificate

Contains:

- Server certificate chain
- Binary DER-encoded X.509 certificates

Purpose:

- Authenticate server identity

Step 5.3: CertificateVerify

Server → Handshake → CRYPTO → CertificateVerify

Purpose:

- Prove ownership of the private key corresponding to the certificate

Step 5.4: Finished (Server)

Server → Handshake → CRYPTO → Finished

Purpose:

- Cryptographic integrity check of the handshake

Step 7: Secure Application Data Transfer

QUIC 1-RTT Data Exchange

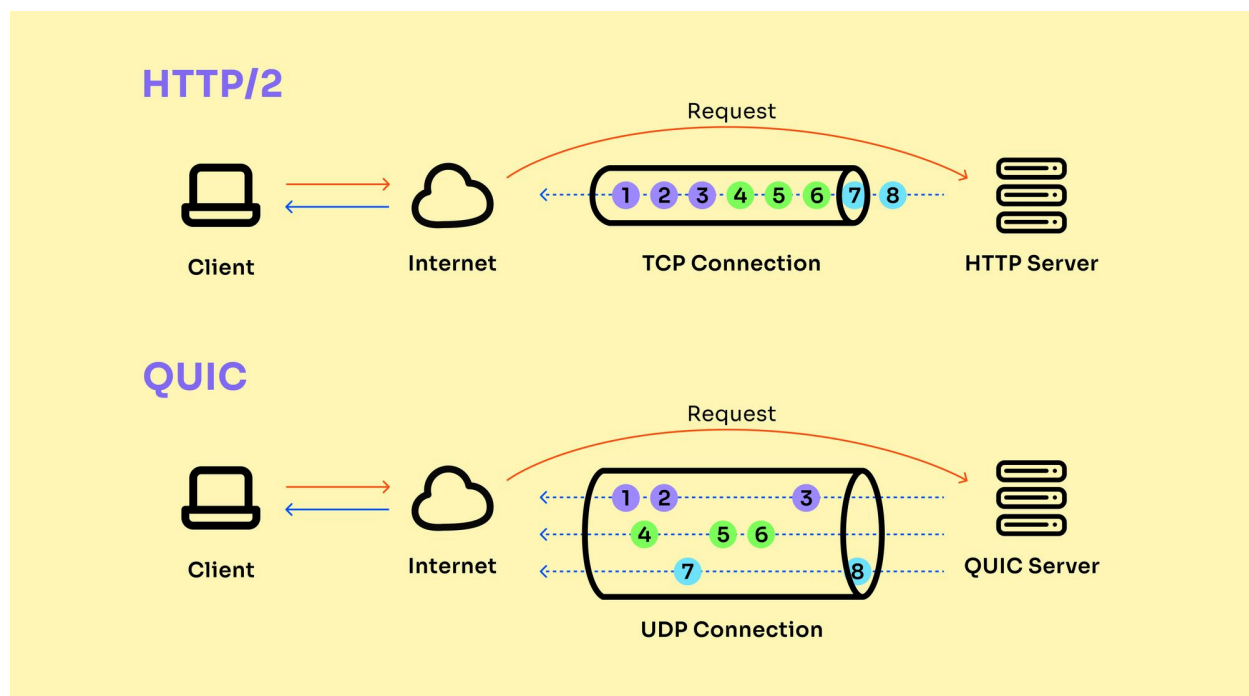
Characteristics:

- Fully encrypted
- Uses 1-RTT keys
- Carries HTTP/3 request and response data

QUIC Payload contains Stream Frame instead of cryptoframe



3. Streams in QUIC



- Multiple streams exist **inside one QUIC connection**
- Each stream is independent of others
- Streams are identified by a **Stream ID**

Ordering and reliability

- Data is:
 - **Ordered within a stream**
 - **Reliable**
- Streams are independent:
 - Loss in Stream A does not block Stream B
- Retransmissions happen **per stream**

Traditional TCP:

- Has **one ordered byte stream**
- Packet loss blocks everything (head-of-line blocking)

QUIC:

- Supports **multiple parallel streams**
- Loss on one stream **does not block others**

4. Why QUIC Uses UDP

Avoids protocol ossification

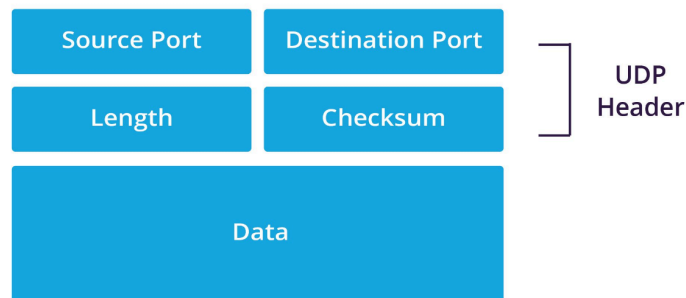
- Middleboxes (NATs, firewalls, load balancers) often inspect TCP headers and enforce strict expectations
- Packets with unfamiliar TCP options or behaviors are frequently blocked, modified, or dropped
- UDP payloads are generally treated as opaque data, allowing QUIC to evolve without interference from middleboxes

Avoids global software and network upgrades

- Introducing a new transport protocol would require:
 - Operating system kernel updates worldwide
 - Updates to routers, firewalls, and other network devices
- UDP is already universally supported across existing network infrastructure, enabling immediate deployment

6. Differences Between QUIC and Generic UDP

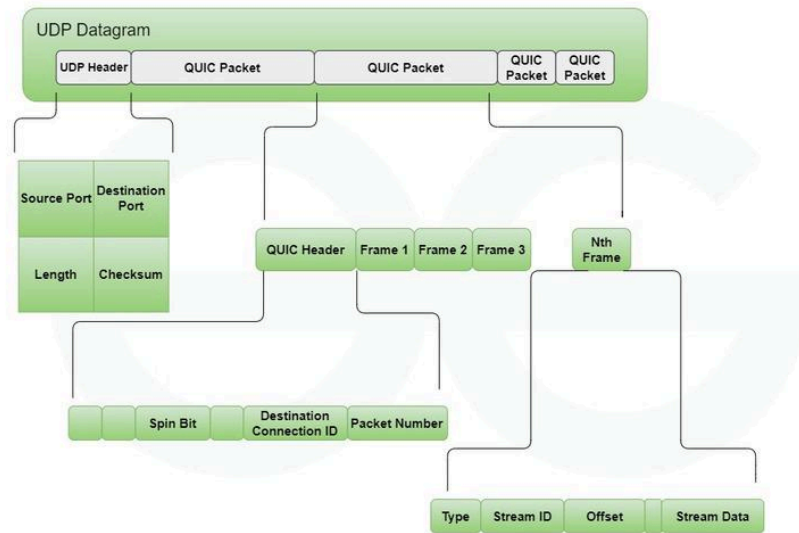
Generic UDP:



- UDP payload has no structure
- Network devices see raw application bytes
- Payload: **fully visible**
- Middleboxes can inspect, modify, or block contents
- No sequence numbers

- No handshake packets
- First packet may already be application data

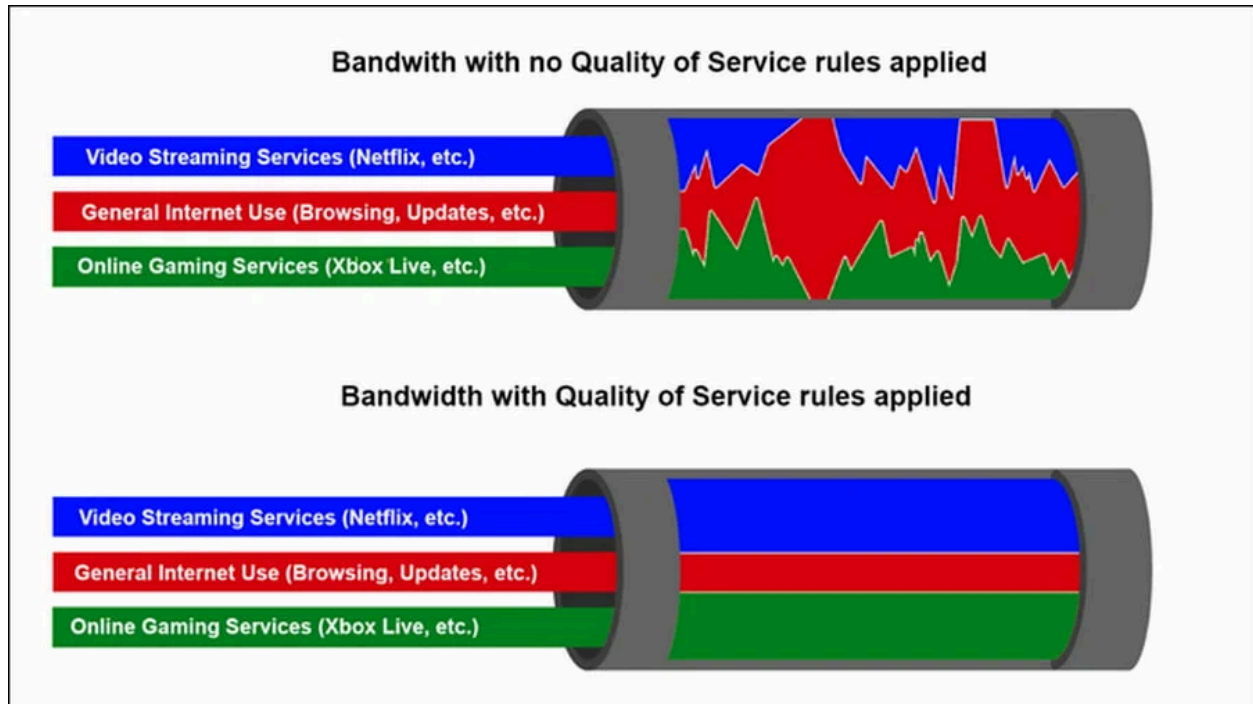
QUIC over UDP



- UDP payload has a defined internal structure
- Payload is not arbitrary data
- Header form (Long / Short)
- Packet type (Initial, Handshake, 0-RTT, 1-RTT)
- QUIC version (Long Header only)
- Destination Connection ID
- Source Connection ID (Long Header)
- Packet length

QOS:

Quality of Service (QoS) refers to a set of traffic management and control mechanisms used in computer networks to prioritize and regulate data transmission based on the requirements of applications.



Types of Quality of Service

1. Stateless Solutions:

- Routers do not maintain detailed information about individual traffic flows.
- This approach is highly scalable and robust, making it suitable for large networks.
- Traffic is treated based on predefined classes rather than per-flow analysis.

2. Stateful Solutions:

- Routers maintain per-flow state information for each traffic stream.
- This allows the network to provide strong QoS guarantees, such as assured bandwidth and controlled delay.

Models:

1. Integrated Services (IntServ)

Integrated Services (IntServ) provides strict QoS guarantees by reserving resources for each individual flow before traffic starts.

- IntServ works on a per-flow basis, so each session can get guaranteed delay, bandwidth, and loss limits.
- Routers keep state information per flow, which increases control but reduces scalability.
- The network uses admission control, meaning new sessions are accepted only if enough resources are available.

2. Differentiated Services (DiffServ)

Differentiated Services (DiffServ) provides QoS by prioritizing traffic classes instead of reserving resources for every flow.

- Traffic is classified into classes (voice, video, best-effort) rather than handled per individual flow.
- Routers manage QoS using aggregated class behavior, which scales better in large networks.
- DiffServ offers relative priority (better treatment for higher classes), not strict end-to-end guarantees.
- Packets are marked (commonly using DSCP) so devices know which class and priority to apply.
- It reduces complexity by avoiding heavy per-flow signaling and per-flow router state.

1. Traffic Classification:

Classification can occur at multiple points in the network:

- End host (OS / NIC)
- Access switch
- Edge router
- Firewall / gateway

1. Header-Based Classification (Stateless)

Classification based only on packet header fields, without keeping any per-flow state.

Fields Used

- Layer 2
 - Source MAC
 - Destination MAC

- VLAN ID (802.1Q)
 - PCP (802.1p)
- Layer 3
 - Source IP
 - Destination IP
 - IP protocol field (TCP/UDP/ICMP)
 - DSCP / TOS
- Layer 4
 - Source port
 - Destination port
 - TCP flags

Table 2. IP Precedence Values and Names

IP Precedence Value	IP Precedence Bits	IP Precedence Names
0	000	Routine
1	001	Priority
2	010	Immediate
3	011	Flash
4	100	Flash Override
5	101	Critical
6	110	Internetwork control
7	111	Network control

2. Deep Packet Inspection (DPI)

Classification by inspecting payload content.

How it works

- Looks for application signatures
- Pattern matching beyond headers

Internal mechanisms

- String matching engines
- Signature databases

Limitations (important)

- Computationally expensive
- Fails with encrypted payloads (TLS, QUIC)
- Often disabled or limited

3. Behavioral / Heuristic Classification

Classification based on traffic behavior, not packet content.

Metrics Used

- Packet size distribution
- Packet rate
- Burst patterns
- Flow duration

2. QoS Policy Mechanism

A QoS policy is a set of deterministic actions applied to packets after classification and before forwarding.

A QoS policy consists of:

1. **Class Map**
Identifies traffic (output of classification)
2. **Policy Map**
Defines actions per class
3. **Attachment Point**
Where the policy is applied (interface, direction)

Marking Actions (Header Modification)

Marking is a **policy action** that modifies **packet headers** to encode priority.

Fields modified:

- IP DSCP
- IP ECN
- VLAN PCP

Policing Actions

Policing is a policy-enforced rate limit.

At the wire level:

- Packet is either:
 - Forwarded
 - Dropped
 - Re-marked

3. Queue Management

Queue management techniques determine how packets are buffered and processed by network devices, impacting the delivery of QoS. Some common queue management strategies include:

- First-In-First-Out (FIFO): The simplest queuing method, where packets are processed in the order they arrive. While easy to implement, FIFO does not consider the importance of different traffic types.
- Priority Queuing: Higher-priority packets are processed before lower-priority ones. For example, voice traffic might be given higher priority than email traffic to ensure smooth communication.
- Weighted Fair Queuing (WFQ): This method distributes bandwidth fairly among different traffic classes based on predefined weights. WFQ ensures that even lower-priority traffic can receive some bandwidth, preventing starvation.

Environment setup:

1. Quiche

- **Required Dependencies:**

Dependency	Purpose	Installation Command
Homebrew	Package manager for macOS	<code>/bin/bash -c "\$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"</code>
Git	Clone quiche source repository	<code>brew install git</code>
Rust (cargo)	Build system and compiler for quiche	<code>curl https://sh.rustup.rs -sSf sh</code>

CMake	Build native dependencies	brew install cmake
pkg-config	Locate system libraries during build	brew install pkg-config
OpenSSL	TLS and crypto library	brew install openssl
curl	Fetch external resources	brew install curl
Wireshark	Packet analysis tool	brew install --cask wireshark
tcpdump	Command-line packet capture	<i>Preinstalled</i>

- **Setup:**

Step No.	Step Name	Description	Command
1	Configure OpenSSL Path	Make OpenSSL visible to Rust build system	export OPENSSL_DIR=\$(brew --prefix openssl)
2	Configure pkg-config	Enable library discovery for OpenSSL	export PKG_CONFIG_PATH="\$OPENSSL_DIR/lib/pkgconfig"
3	Clone quiche Repository	Fetch quiche source code with submodules	git clone --recursive https://github.com/cloudflare/quiche.git
4	Enter quiche Directory	Move into quiche source directory	cd quiche
5	Build quiche	Compile quiche client and server binaries	cargo build --release

6	Create Certificates Directory	Prepare directory for TLS certificates	mkdir certs
7	Generate TLS Certificate	Generate self-signed certificate for server	openssl req -x509 -newkey rsa:2048 -keyout certs/key.pem -out certs/cert.pem -days 365 -nodes -subj "/CN=localhost"
8	Start quiche Server	Launch QUIC server over UDP	cargo run --release --example server -- --cert certs/cert.pem --key certs/key.pem --listen 127.0.0.1:4433
9	Start quiche Client	Initiate QUIC connection to server	cargo run --release --example client -- https://localhost:4433
10	Verify UDP Listener	Confirm server is listening on UDP port	netstat -an grep 4433

Packet(pcap) Analysis using wireshark:

Network traffic analysis tool showing a list of captured packets. The interface includes a toolbar at the top, a packet list table with columns for No., Time, Source, Destination, Protocol, Length, and Info, and a detailed packet view on the right. The selected packet is a QUIC packet (No. 711) from 2405:201:5804:115f:: to 2404:6800:4002:81a::. The packet details show it is a Protected Payload (KP0) with a length of 5268 bytes. The packet is part of a stream with ID 0 and is the first packet in the stream. The packet is encrypted and contains a protected payload (KP0).

Network traffic analysis tool showing a detailed view of a QUIC packet. The interface includes a packet details pane on the left, a packet data pane in the center, and a packet hex dump pane on the right. The selected packet is a QUIC packet (No. 711) from 2405:201:5804:115f:: to 2404:6800:4002:81a::. The packet details show it is a Protected Payload (KP0) with a length of 5268 bytes. The packet is part of a stream with ID 0 and is the first packet in the stream. The packet is encrypted and contains a protected payload (KP0).

Dataset:

1. A large one-month QUIC network traffic dataset from backbone lines

Link: <https://zenodo.org/records/10728760>

- ID: Unique identifier
- SRC_IP: Source IP address
- DST_IP: Destination IP address
- DST_ASN: Destination Autonomous System number
- SRC_PORT: Source port
- DST_PORT: Destination port
- PROTOCOL: Transport protocol
- QUIC_VERSION QUIC: protocol version
- QUIC_SNI: Server Name Indication domain
- QUIC_USER_AGENT: User agent string, if available in the QUIC Initial Packet
- TIME_FIRST: Timestamp of the first packet in format YYYY-MM-DDTHH-MM-SS.ffffff
- TIME_LAST: Timestamp of the last packet in format YYYY-MM-DDTHH-MM-SS.ffffff
- DURATION: Duration of the flow in seconds
- BYTES: Number of transmitted bytes from client to server
- BYTES_REV: Number of transmitted bytes from server to client
- PACKETS: Number of packets transmitted from client to server
- PACKETS_REV: Number of packets transmitted from server to client
- PPI: Packet metadata sequence in the format: [[inter-packet times], [packet directions], [packet sizes]]
- PPI_LEN: Number of packets in the PPI sequence
- PPI_DURATION: Duration of the PPI sequence in seconds
- PPI_ROUNDTRIPS: Number of roundtrips in the PPI sequence
- PHIST_SRC_SIZES: Histogram of packet sizes from client to server
- PHIST_DST_SIZES: Histogram of packet sizes from server to client
- PHIST_SRC_IPT: Histogram of inter-packet times from client to server
- PHIST_DST_IPT: Histogram of inter-packet times from server to client
- APP: Web service label
- CATEGORY: Service category
- FLOW_ENDREASON_IDLE: Flow was terminated because it was idle
- FLOW_ENDREASON_ACTIVE: Flow was terminated because it reached the active timeout
- FLOW_ENDREASON_OTHER: Flow was terminated for other reasons

